Intensive Study on Various Controllers for Two-wheeled Robot and Car like Mobile Robot

Manahil Shahid[1], Karina Jaalouk[2], Yunus Salih[3]

Abstract

This paper is a detailed overview of various type of controllers, their design, working principle, and how they work with various kinds of robot. The main robots under study were differential drive robot and car-like mobile robot. The controllers under study were Nominal Controller, Stanley Controller, and Model Predictive Controller. The main aim of the comprehensive study is to provide the practical implications of the existing controllers and how they can be integrated to work in a systematic way for the streamlined working of the robots. The case understudy are comprised of the following case scenarios. The discussion of the Nominal and Model Predictive Controller with the differential drive robot and Stanley controller for the Car like robot. Additionally, a case study regarding the working behavior of Model Predictive Controller for the Obstacle Avoidance in the environment comprising of various obstacles is conducted to analyze and visualize the behavior of the robot and how it avoids the high cost area. A validation study is discussed in the later part of the report showcasing the impacts of various parameters such as length of the horizon (nactor), dt, prediction step, and diagonal matrix R. The cost function is also designed and implemented for the MPC. Lastly, there is a discussion about the design of experiments, results, and verification study explaining the trends and impact of various parameters.

Nominal Controller, parameters, Stanley Controller, Model predictive Controller MPC, horizon length

1 Introduction

Mobile robotics is a rapidly advancing field with applications ranging from autonomous vehicles to industrial automation and service robots. Enabling robots to navigate and function well in their surroundings is the main objective of mobile robot control. This entails creating control algorithms that guarantee the robot stays stable, avoids obstructions, and follows the intended course. The rising need for autonomous systems capable of dependable operation in dynamic and unexpected contexts is the driving force behind the project. Advanced control algorithms on mobile robots can improve efficiency, productivity, and safety in a number of industries, including logistics, medical, and transportation. This study seeks to determine the best methods for mobile robot navigation by investigating and contrasting various control strategies, ultimately assisting in the creation of more resilient and adaptable robotic systems.

The main aim of this extensive study was to understand the key concepts of control theory, including various controllers and their pros, cons, usability, adversity and liability such as nominal and Stanley controller. Most importantly the main focus was the formulation of optimal control problem solution. In this particular case the controller understudy is Model Predictive Controller that tends to provide the most optimal solution for path planning and identifying the sequence of optimal control inputs. The main aim of this study is to project the behavior of two types of mobile

robot; differential mobile robot and car like mobile robot, using nominal and Stanley controller. The first section of the robot is a brief introduction of Differential Drive and Car-Like mobile robot. It briefly explore the mathematical models and main idea behind the models. The motion control theory via path planning and trajectory formulation are the main areas of the study. The mobility of a mobile robot is governed by the constraints imposed on it. These constraints are typically derived from the type of wheels present on the robot selected while constructing the robot depending on its maneuverability and application based requirements.

This report includes implementation and detailed analysis of Nominal, Stanley, and Model Predictive Controller known as MPC. Moreover, various experiments were conducted to analyze the behaviour of these controllers under different changing parameters such as tuning parameters, initial robot pose, goal position and many more. The later half of this report also discuss about the optimal control solution implemented via Model Predictive Controller. This section also includes the extensive study on how the controller works and the impact on its performance by changing various parameters such as horizon length, matrix R, and time step. Also, the formulation of cost function and correlation between the penalties and performance of the MPC. The obstacle avoidance capabilities of MPC are also part of the research methodology. Two approaches were used, first an environment containing the obstacles were created as a new class to study the impact on the performance of MPC in the presence of obstacles and how it performs while selecting the optimum path for its motion avoiding the obstacles. The main objective of MPC is to minimize the overall cost function and maximize the optimal performance. In the second approach multivariate normal distribution is used to detect the high cost regions and optimize the overall performance of MPC. The later sections of the report further explore the some case studies where the performance of the controllers on turtle-bot is visualized in Gazebo and real world environment.

2 Systems

This section explores the two main systems that are used for the formulation and design of the controller. It discusses about the mathematical models, working criteria, and major constraints. The two main systems are; Differential Derive Mobile Robot, and Car-Like Mobile Robot.

2.1 Differential Drive Mobile Robot

A differential drive mobile robot is characterized by its two independently driven wheels, which allow it to perform in-place rotations. Because of its easy agility and straightforward mechanical design, this kind of robot is commonly employed in many mobile robots used now a days. A differential mobile robot must be controlled by knowing which wheel speeds to use in order to follow the intended path. It is basically comprised of two wheels on a common axis and each of the wheel is capable of moving in forward and backward direction independently. The differential derive robot is capable of rotating about the point that is common along both of its right and left wheel is called as *ICC* – Instantaneous Center of Curvature. The velocity of left and right wheel can be varied to make robot undergo a rolling motion.

2.1.1 Mathematical Model

The mathematical model of a non-holonomic two wheeled differential derive mobile robot can be described by its kinematic and dynamic model. In this report our main

focus will be on the kinematic model of the mobile robot, therefore, it will be explained and discussed in detail. Following block diagram shows the overall scheme that represents a differential drive mobile robot:

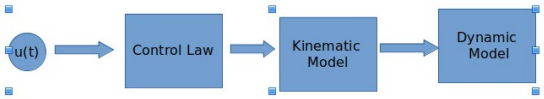


Figure 1: Block Diagram

The input of the controller is the desired velocity of the robot *a(t)*, whereas *q(t)* is the current velocity of the robot that is later transformed by the kinematic model into the robot's pose. The control input is the difference between the robot's desired velocity and the current velocity, and the output of the controller impacts the dynamics of the mobile robot, such as force or torque. The pose of the robot can be expressed with respect to the reference coordinate system in its local coordinate system by x and y, and the rotation is defined by angle θ. The rotational position of the robot wheels is defined by θ_R and θ_L. The position vector of the robot can be described as p:

$$P_R = \begin{bmatrix} x \\ y \\ \theta \\ \theta_R \\ \theta_L \end{bmatrix} \tag{1}$$

The kinematic model of the robot can be expressed by it linear and angular velocity. The position of the robot in reference frame with respect to its position in local frame can be expressed using homogeneous transformation as shown in the equation:

$$P'_k = R \cdot P_R \tag{2}$$

Here the rotation matrix can be derived as follows for the given positive counter clockwise rotation:

$$R = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{3}$$

The next section of this report will discuss about kinematic model of the differential derive mobile robot in detail.

2.1.2 Kinematic Model

The Kinematic model of the differential mobile robot as shown in the figure 2 can be expressed by its linear and angular velocities.

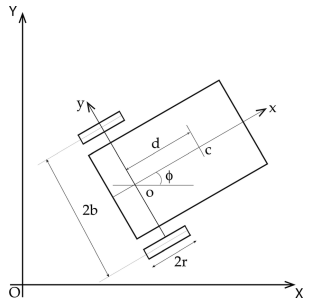


Figure 2: Differential Drive Robot

Following equation gives the complete description of the system as shown below:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \tag{4}$$

Here

$$\dot{x}$$

and

$$\dot{y}$$

are the linear velocities of the robot with respect to the global reference frame. For explaining the overall control of the mobile robot it is often necessary to describe the role of the wheel velocities to define the complete kinematic model of the differential mobile robot. The overall linear velocity of the robot can be calculated by computing the mean of its right and left wheel velocities as given in the equation

$$v = \frac{V_R + V_L}{2} \tag{5}$$

The angular velocity can be expressed as follows:

$$\omega = \frac{\omega_R - \omega_L}{2b} \tag{6}$$

where *2b* is the distance between the two wheels. Since, the relationship between the linear and angular velocities is given as

$$V = r\omega \tag{7}$$

Therefore, the relationship can be expressed as

$$V = r\frac{\omega_R - \omega_L}{2b} \tag{8}$$

Hence, above defined mathematical equation gives the complete control representation of two wheeled differential drive mobile robot with non-holonomic constraints for our control applications using Nominal controller and Model Predictive controller.

2.2 Car-Like Mobile Robot

Car-like mobile robots, also known as Ackermann steering vehicles, mimic the steering mechanism of cars. Because they include driving wheels and a steering wheel, they may be used in applications like autonomous automobiles that need for motion that is natural and fluid. A car-like robot must be controlled by adjusting its steering angle and speed in order to follow a predetermined route. Car like mobile robots are extremely common because of their better usability, stability, and vast range of applications in the autonomous robotics. The car-like mobile robots are the main focus of research now-a-days due to the fact that automating vehicles has become a trend due to their several remarkable features and advantages. The mathematical model of the Car-like mobile robot can be divided into two main sections; Kinematic and Dynamic Model. This report is particularly focusing on the kinematic model of the system to design the Stanley controller for steerable cars. The model of the car-like mobile robot is shown in the figure below:

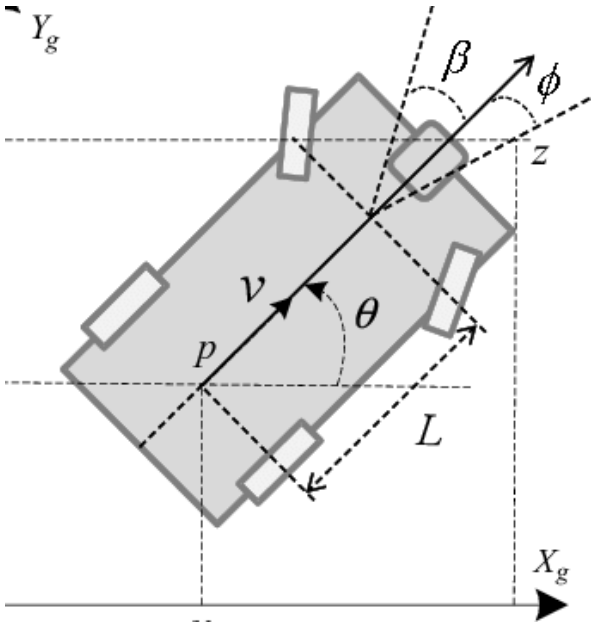


Figure 3: Car Like Model Robot

2.2.1 Kinematic Model

In order to derive the mathematical model of the Car-like mobile robot with non-holonomic constraints. The figure 2 given below showcase the representation of a car-like mobile robot in a given coordinate system. Non-holonomic constraints are defined to be the constraints that are not integrable, therefore the general mathematical expression expressing such constraints can be defined as follows:

$$\dot{u} \sin\theta + \dot{\omega} \cos\theta = 0 \tag{9}$$

where

$$\dot{u}$$

and

$$\dot{\omega}$$

are the velocities of the wheels with respect to the given frame. For the sake of simplification, lets assume that the steering angles are very small and in that case the model

can be replaced by the model of a bicycle as shown in the figure 3. Here, x and y gives the position of the system with respect to origin in a given coordinate system where as

$$\theta$$

is the angle specifying the orientation of the system w.r.t to x-axis as specified in the figure and

$$\phi$$

is the steering angle between the steering axis and the body of robot. Let a and b be the distance of front and rear axle from center of gravity so the distance between the front and rear wheel can be expressed as $L=a+b$. Let the position of the front and rear axle described by the coordinates *(x1, y1)* and *(x2, y2)* respectively. So, that can be mathematically computed as follows using the graphical interpretations, For the position of the front axle:

$$x_1 = x - b\cos\theta$$
$$y_1 = y - b\sin\theta$$

Similarly, for the rear axle the equations can be derived as:

$$x_2 = x + a\cos\theta$$
$$y_2 = y + a\sin\theta$$

Now, taking the derivative of the above mentioned equations will give:

$$\dot{x}_1 = \dot{x} + b\sin\theta\,\dot{\theta}$$
$$\dot{y}_1 = \dot{y} - b\cos\theta\,\dot{\theta}$$

Similarly, doing the same for the rear wheel:

$$\dot{x}_2 = \dot{x} - a\sin\theta\,\dot{\theta}$$
$$\dot{y}_2 = \dot{y} + a\cos\theta\,\dot{\theta}$$

The equation of the non-holonomic constraints defined above then becomes:

$$\dot{x}_1\sin\theta - \dot{y}_1\cos\theta = 0$$

$$\dot{x}_2\sin(\theta+\phi) - \dot{y}_2\cos(\theta+\phi) = 0$$

Therefore, substituting the values of $\dot{x}_1$, $\dot{x}_2$, $\dot{y}_1$, and $\dot{y}_2$ gives the system equation in terms of x and y. Hence, the equation becomes:

$$\dot{x}\sin\theta - \dot{y}\cos\theta + b\dot{\theta} = 0$$

$$\dot{x}\sin(\theta+\phi) - \dot{y}\cos(\theta+\phi) + a\dot{\theta}\cos\theta = 0$$

Therefore, by solving these equation by translating the equations using the coordinate axis of the robot where u_u and u_w represent velocities along the u-axis and w-axis, respectively,

$$\dot{x} = u_u\cos\theta - u_w\sin\theta$$
$$\dot{y} = u_u\sin\theta + u_w\cos\theta$$

Now, substituting them in the previous final equation gives:

$$\dot{u_w} = b\dot{\theta}$$
$$\dot{\theta} = u_u \tan\phi / L$$

Now, considering v and w as the linear and angular velocities, the whole system can be expressed as:

$$\dot{x} = v\cos\theta$$
$$\dot{y} = v\sin\theta$$
$$\dot{\theta} = \omega$$
$$\omega = v/L\tan\phi$$

This is the mathematical representation of the Car-Like mobile robot.

3 Methodology

This section explores the methodologies designed to implement all the three controllers, the design idea, and their integration with the system model. For this particular study two main models were considered; Two-wheeled differential drive robot and Car -like mobile robot that are explained in the above section. This section briefly explains the research conducted and a deep overview of the basic working principles extending to the implementation of the various controllers for the defined systems. In this section the design of controllers will be discussed along-with their implementation with the models discussed above. The main objective of this section is to define the sections in rcognita.edu for each of the controller, Nominal, Stanley and Model Predictive Controller.

3.1 Nominal Controller

A Nominal Controller is the simplest yet most effective controller for driving and controlling the mobile robot. It is important to be noted that the control of the mobile robots is not as simple as it sounds. It requires a close consideration of various parameters such as tuning of hyper-parameters pho, alpha, and beta that are designed for the nominal controller and will be discusses shortly in this section. Lets first discuss the three major control variables in the case of nominal controller. The three control variables are x and y position of the robot and theta that is the orientation of the robot with two control inputs linear velocity v and angular velocity w . The main aim is to design a control algorithm to control the movement of the mobile robots to reach the desired goal position. For the implementation of the nominal controller consider the differential mobile robot whose model was explained in the section 3.1. Consider a mobile robot in a world coordinate system as shown in figure 4. The equations given below describes the mathematical model of the differential derive robot:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}$$

Let x_g and y_g be the position of the goal, and x_r and y_r be the position of the robot. The control input in this case are linear and angular velocity v and w. The coordinate transformation gives the following equations with respect to the origin at its goal position.

$$\rho = \Delta x + \Delta y$$
$$\alpha = -\sqrt{\theta + \text{atan2}(\Delta y, \Delta x)}$$
$$\beta = -\theta - \alpha$$

These equations are implemented using the python code inside the class Nominal Controller. These gives the description of robot in a newly defined coordinate system. Since alpha is the angle between the robot x-axis and the vector joining the rear of robot with the goal position, it is defined as follows: For $\alpha \in I_1$, the forward direction of the robot points toward the goal, while for $\alpha \in I_2$, it is the backward direction. By properly defining the forward direction of the robot at its initial configuration, it is always possible to have $\alpha \in I_1$ at $t = 0$. However, this does not mean that α remains in I_1 for all time. Therefore the control inputs v and w are defined as:

$$v = \kappa_p \rho, \quad \omega = \kappa_\alpha \alpha + \kappa_\beta \beta$$

Here,

$$\kappa = (\kappa_p, \kappa_\alpha, \kappa_\beta)$$

are the tuning parameter and must be fine tuned unless optimal solution is achieved. While setting the tuning parameters following constraints must be considered: $\kappa_\rho > 0, \kappa_\beta < 0, \kappa_\alpha - \kappa_\rho > 0$

In this case various experimentation were conducted and the selected optimal values are as follow:

$$\kappa = (\kappa_\rho, \kappa_\alpha, \kappa_\beta) = (0.15, 0.17, -0.05)$$

The following figure 4 shows the animations of the differential derive mobile robot with Nominal controller implemented in rcognita:

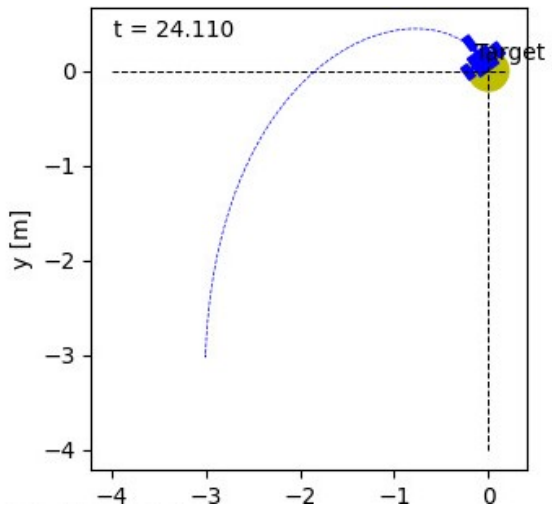


Figure 4: Nominal Controller Trajectory

This figure shows the trajectory followed by the robot and its linear and angular velocity. The graphs shown in the in the figure 5 below also tracks the movement of the robot and showcase the distance travelled during each time step until it reaches the goal.

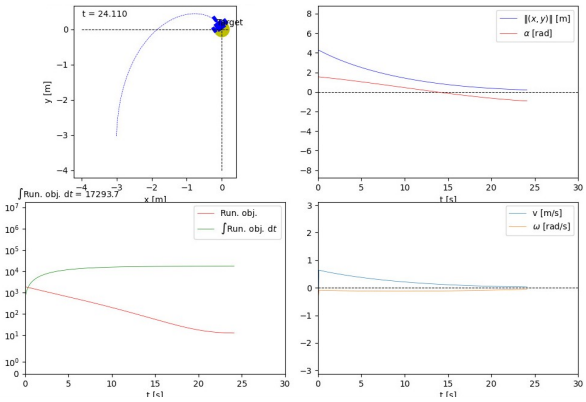


Figure 5: Performance Graphs

3.2 Stanley Controller

Stanley controller provides the most effective approach for path tracking control in car like mobile robots. It simplifies the Ackeramann steering model into bicycle model for geometric path tracking. Therefore, the model is simplified by combining the front wheels as one and rear wheels as one forming a two wheeled model also the other simplification was non-holonomic constraints that are discussed in detail in section 3.2. These simplifications provide a relationship between the steering wheel and the curve followed by the rear wheel. Therefore, the steering angle phi as shown in the figure 6 can be defined as:

$$\tan^{-1}\phi = \frac{L}{R} \tag{10}$$

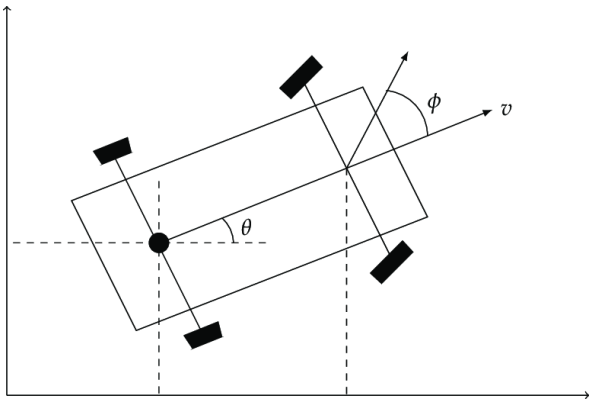


Figure 6: Car Model

Here L is the length between the two wheels and R is instantaneous radius of the curvature. The controller tends to calculate the look ahead distance and plan its course of motion to follow the trajectory. The method involves calculating the curvature of a circular arc that links the rear axle's current position to a target point on the path in front of the vehicle. This target point is established by measuring a look-ahead distance ℓ_d from the current position of the rear axle to the desired path. For the implementation of the Stanley controller for trajectory tracking first of all a trajectory is generated by defining a new class called Trajectory Generator inside the controller module. Various trajectories were created as shown in the figures below:

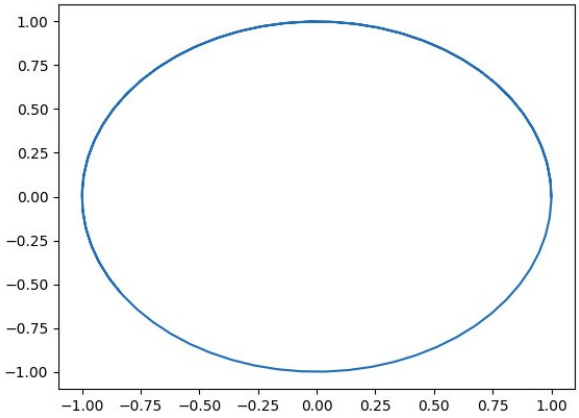


Figure 7: Circular Trajectory

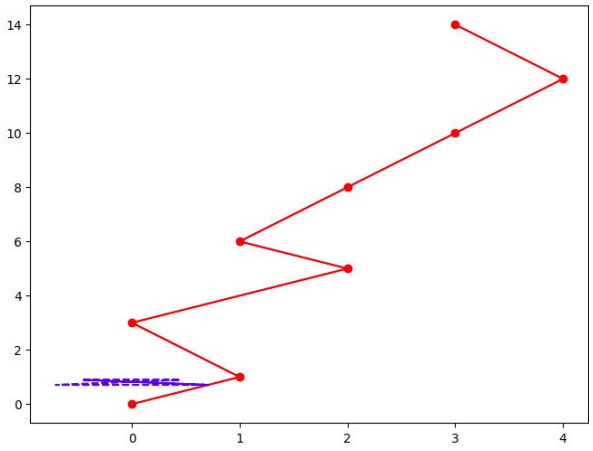


Figure 8: Zigzag Trajectory

The circular trajectory was created inside the module using the way point technique as shown in figure 7. It is used for testing of the Stanley controller for that the robot can follow the path. Stanley controller unlike the other traditional controller does not work with the real axle distances rather it works with the position of the front axle. In addition correcting the theta error, i also computes the distance between the front wheel and the closest point on the trajectory and applies the correction. This is normally called as the cross track error.

$$\theta_e = \theta - \theta_p \tag{11}$$

In the control algorithm, first it calculates the position of the front wheel of the robot then The distances (`dx` and `dy`) between the front wheel and the current goal way point are calculated. The Euclidean distance (`dist_to_goal`) is then determined. Then a condition was set that if the robot is within the area of 0.8 m from the current goal way point the goal way point is incremented to the next way point and in this way the robot moves to next point and follow the trajectory. In order to follow the trajectory and keep the robot on path the orientation error is computed as the difference between the desired orientation and the robot's current orientation.

Now, to update the current state of the vehicle steering angle (`delta`) is calculated using this error and the lateral error returned by the `error` function. The error function is used to compute the error for adjusting the steering angle that is actually the

perpendicular distance from the robot to the desired path. This computes actually how far is the robot from the desired path so that it can adjust its orientation and position accordingly. Lastly the linear and angular velocities of the robot are computed. Following equations are implemented: Steering angle in figure 9:

$$\delta(t) = \theta(t) + \arctan\left(\frac{\kappa e_f(t)}{v(t)}\right)$$

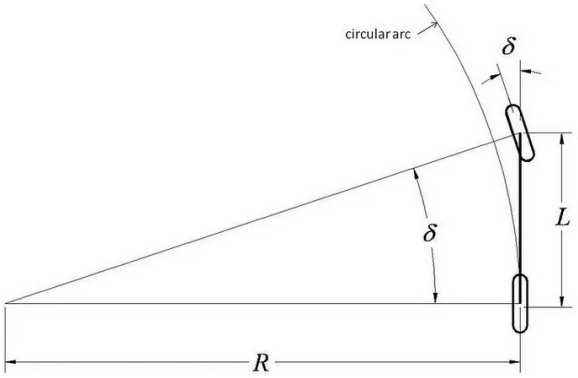


Figure 9: Equivalent Car Model

Here kappa is the controller gain that needs to be adjusted to smooth the trajectory following and finding the optimum value that gives the most accurate result. There is always a trade off between the tracking performance and stability therefore, look ahead distance calculation plays a very important role in determining the stability of the robot, it must not be very large nor too small. The error is defined by the following given equation:

$$\text{error} = (y_{\text{goal}}(t) - y(t))\cos(\theta(t)) - (x_{\text{goal}}(t) - x(t))\sin(\theta(t)) \tag{12}$$

The distance to the goal is calculated using following equation:

$$\text{distance to the goal point} = \sqrt{(y_{\text{goal}}(t) - y(t))^2 + (x_{\text{goal}}(t) - x(t))^2} \tag{13}$$

The control gain can be adjusted accordingly. If we make the control gain too high the robot does not follow the trajectory and looses the stability as shown in the figure 10 given below:

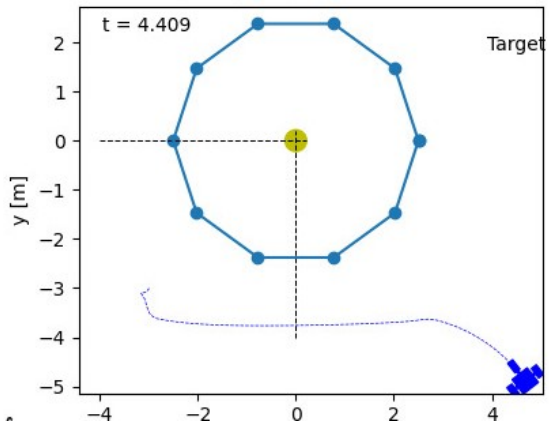


Figure 10: Trajectory with High Controller Gain

Also, if the control gain is set so low such as 0.5, in that case also robot does not follow the trajectory properly. This case can also be visualized in the figure 11 shown below:

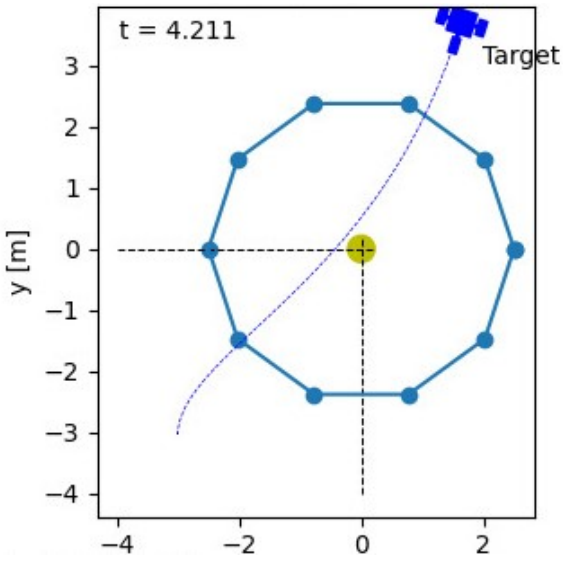


Figure 11: Trajectory with Low Controller Gain

Therefore, the optimal control gain value that was selected after the series of experimentation and several hit and trial method is 3 as the result shown below in the figure 12 given below:

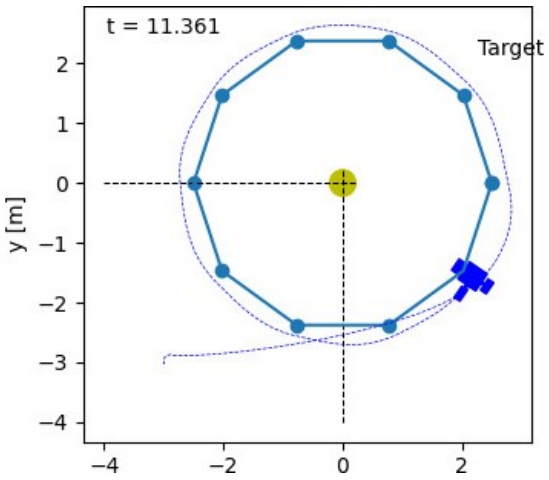


Figure 12: Trajectory with Optimal Controller Gain

3.3 Advanced: Model Predictive Controller

Model Predictive Controllers are the most advanced solution for the optimal control problem. Due to its capability to manage constraints and potentially nonlinear dynamics, Model Predictive Control (MPC) has emerged as a leading modern control method in industrial applications. It tends to minimize the cost function by maximizing the controller's optimal performance. It is designed to determine the sequence of optimal control inputs to optimize the control of the linear and non-linear dynamic systems. It is also sometimes referred as the receding horizon control. To simply put, the main purpose of the MPC is to plan the series of control actions depending upon the predictive model of the system to minimize the cost function over a finite time horizon. Therefore, the controller tends to update the state of the system depending on the feedback and predicting the most optimal solution. This can be visualized as the goal objective of a car to reach a goal position as quick as possible which means that it has to decide the shortest path that will take less time and each available path has a certain cost associated with it therefore, it will select the path with the minimum cost therefore, declared as the optimal path.

Model Predictive Control Solution can be regarded as the real time repetitive optimization that chooses the optimal sequence of control actions and perfectly solve the problem of optimization. In the real world, all physical systems have constraints such as performance, safety, Input, output constraints and many more and classical controllers often ignore the presence of constraints the real world and only consider ideal scenarios but predictive controller resolves such kinds of issues by providing better constraint satisfaction and generating optimal control function. Mathematically, MPC control can be defined a:

$$u^*(x(t)) = \arg\min \sum_k Q(\mathbf{x}_{t+k}, \mathbf{u}_{t+k}) \tag{14}$$

Here, *x(t)* defines the state of the system and *u(t)* describes the control input that are subject to constraints as

$$u_{t+k} \in U, \quad x_{t+k} \in X \quad \text{for} \quad k = 0, \ldots, N-1 \tag{15}$$

$$x_u(N) \in X_0 \tag{16}$$

Here U is defined as the input constraints, X as the input constraints and the terminal constraints.

3.3.1 Optimal Control Problem Formulation

MPC solves the optimization problem and provides the solution at each time step by finding the sequence of optimal control actions that minimizes the cost function. The solver mainly considers the initial state at the start of the optimization problem, prediction horizon that predicts and optimizes the future trajectory of states and inputs, and lastly cost which it tends to minimize over the entire prediction horizon to avoid high cost region and provide optimal control actions. In order to formulate an optimal control problem following are the main aspects that define the control problem:

1. Objective to minimize the overall cost function
2. Definition of the Internal System of the model
3. Physical Constraints(Input, Output, Linear, Quadratic)

The main objective while defining the optimal control problem solution is to minimize the cost function. The cost function can be defined as follows:

$$y_o(\mathbf{x}(o), \mathbf{u}(o)) = \mathbf{x}_N^\top P \mathbf{x}_N + \sum_k \mathbf{x}_k^\top Q \mathbf{x}_k + \mathbf{u}_k^\top R \mathbf{u}_k \tag{17}$$

Here R, P, and Q are the weight matrices and are greater than zero. In this case the robot tends to follow the shortest path by minimizing the cost function. There is a continuous feedback that predict the current state of the system at each time step and find the optimal control sequence for entire planning window and implement it over the given time horizon and the update the system current state. This process happens in an iterative manner depending upon the selected horizon length. In the case of using Model Predictive Controller for path tracking and reaching the goal in minimum time the most intuitive approach is to move forward and plan the path while making careful and informed decision while choosing the paths to minimize the overall cost function. The later half of this section explores the parameters on which the performance of the MPC depends upon and how they influence the overall performance of the robot.

The parameters that greatly influence the performance of the controller and whose impact will be later studied to analyze their impact on the controller's performance:

1. **Nactor**: This parameter sets the length of the prediction horizon, determining how far into the future the controller plans the sequence of actions. The performance was analyzed by setting the prediction horizon to 1, 6, and 15.
2. **Gamma**: The discounting factor used in the cost function. It determines how much weight is given to future rewards or costs.
3. **dt**: The prediction step size that impacts the sampling time and how quickly the robot reaches the goal position.
4. **Running Objective**: This function determines the running cost and the structure of the objective function.
5. **Sampling Time**: The time interval between consecutive controller updates or samplings.

Therefore, the behavior of these parameters will be studied.

3.3.2 Cost Function

Cost function is also referred as an objective function. It is defined as a mathematical function that determines the cost or penalty related to certain set of actions in a given system as in our case it is related to the path followed by the robot to reach the goal position. Normally, the straight path is the shortest path and in this case it works the same the robot tends to move across the diagonal straight line to minimize the overall cost function. In control theory (such as Model Predictive Control), the cost function quantifies the deviation from desired states or control inputs over a horizon. In this case quadratic cost function is implemented to analyze the working performance of the controller. In this case two cost functions are implemented: 1. Quadratic Cost Function This cost function is given as follows:

$$\chi^\top R_1 \chi \tag{18}$$

Here *R1* is the square weight matrix and chi is a column vector. The dimension of both are same and the final result of the multiplication is a scalar quantity giving the cost. The result of robot's motion using MPC with quadratic function is shown in the figure 13 below:

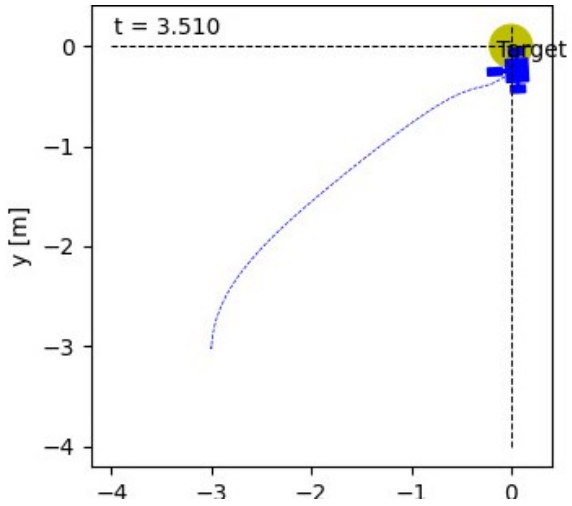


Figure 13: Quadratic Cost Function

2. Bi-quadratic Cost Function The bi-quadratic cost function is implemented using the following mathematical equation:

$$\left(\chi^\top R_2 \chi\right)^2 + \chi^\top R_1 \chi \tag{19}$$

Therefore, *R1* and *R2* are the weight matrices. The results visualized with the MPC with bi-quadratic function are shown in the figure given below 14:

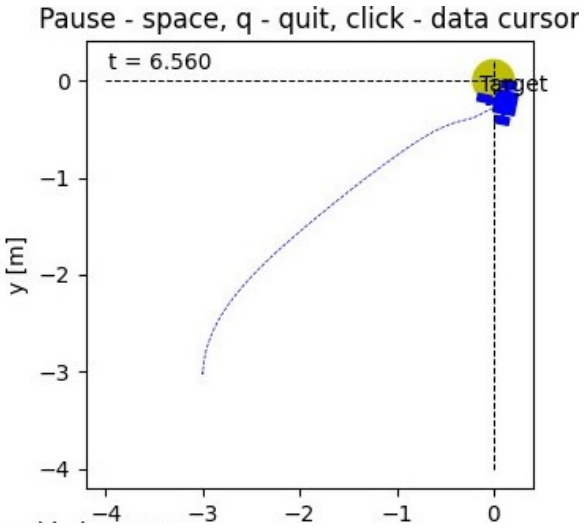


Figure 14: Bi-Quadratic Cost Function

It can be clearly seen that quadratic cost provides most optimal solution as the robot chooses most straight-forward and the shortest path yet minimizing the cost and maximizing the controllers overall performance.

3.3.3 Avoiding High Cost Region

The high cost region is avoided by the optimal design of cost function, constraints, and various optimization strategies such as receding horizon approach. The main objective of the cost function to penalize the undesired control actions and constraints prevents the system from violating the safety limits as all the real world system have constraints so they needs to be modelled accurately. In Model Predictive Control (MPC), avoiding high-cost regions means to ensuring the condition that the desired control actions keep the system's states and inputs away from areas where the cost function is high. This is important for maintaining system performance and efficiency. Therefore, this feature make MPC the most advance and optimal controller version. For the implementation of the high cost region multivariate normal distribution is used. A separate function is defined within the class controller optimal predictive to implement high cost region. This determines the probability of the high cost region and MPC algorithm tries to find the most optimal path that reaches the goal quickly and in most straightforward manner. Mathematically, the probability density function of multivariate normal distribution is defined as follows:

$$f(x) = \frac{1}{(2\pi)^{d/2}\sqrt{\det(\Sigma)}} \exp\left(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\right) \tag{20}$$

Here,

1. x is a d--dimensional column vector representing the point at which we want to evaluate the density.
2. μ is a d-dimensional column vector representing the mean of the distribution.
3. Σ is a covariance matrix of the distribution.
4. $det(\Sigma)$ denotes the determinant of the covariance matrix.

The following expression computes the quadratic form in the mathematical equation for multivariate distribution defined above::

$$\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu) \tag{21}$$

Therefore, this helps in analyzing the high cost regions and algorithms works to avoid the high cost regions. In the later section of this report the impact on the performance of the controller is analyzed be changing different parameters and relationship between them.

4 Validation

This section will explore the design of experiment along with the comparison of various results that were achieved by performing the extensive experimentation throughout the semester using rcognita, Gazebo, and turtle-bot in the real world setting which in this case is the classroom environment.

4.1 Design of Experiment for MPC

For testing the performance of MPC, the primary goal in this study is focused on three parameters i.e. Horizon length (Nactor), discounting factor (gamma), and time step (dt). Following performance metrics are analyzed during the experiments:

1. Stability over the simulation period.
2. Shortest path taken.
3. Time taken to compute control actions.
4. Accumulative cost at the end of the horizon length.

Each parameter was changed twice to study the impact on the performance metrics. Following range of values for each of the parameter is selected:

1. **Horizon length:**[6, 15]
2. **Discounting factor: [0.7, 0.9]**
3. **Time step: [0.1s, 1.0s]**

According to full factorial design 2x2x2 = 8 set of experiments were performed to analyze the relationship or dependence between each parameter and how it has impact the overall performance metrics of the controller. So, each parameter was set according to the chosen values and then result was recorded by running the MPC simulations. The output data was stored in the form of log files and then the results were plotted. The first experiment was conducted by changing length of horizon and keeping gamma and time step constant. Nactor = 6, gamma = 0.9, prediction step size = 0.1 Following graphs in figure 15 shows the relationship between time and velocity and velocity and total accumulative cost by changing the length of horizon:

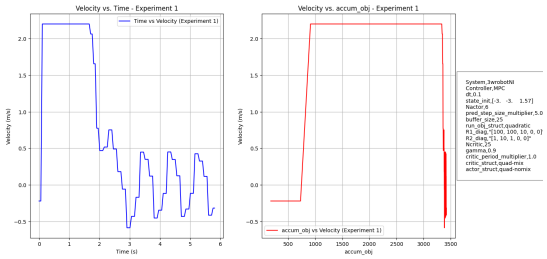


Figure 15: Experiment 1

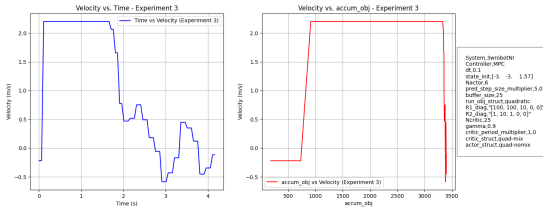


Figure 16: Experiment 4

Now, again Nactor is increased and other two factors are kept constant. The graph shown below in figure 16 is the impact on the velocity tracking of the robot when horizon length was increased therefore, better prediction and better velocity tracking.

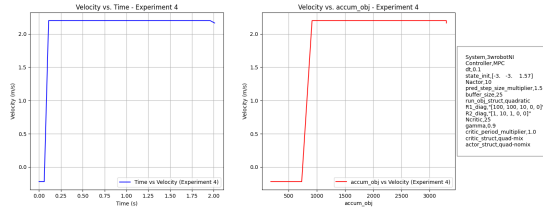


Figure 17: Experiment 2

From these two graphs we can interpret that by increasing horizon length controller is making more informed decision therefore, resulting in more smooth performance and optimal velocity profile. The overall accumulative cost is minimized.

Now, Discounting factor gamma is changed and all other two factors are kept constant. Firstly, gamma is set to be 0.1 and then 0.9. Following two graphs show the result.

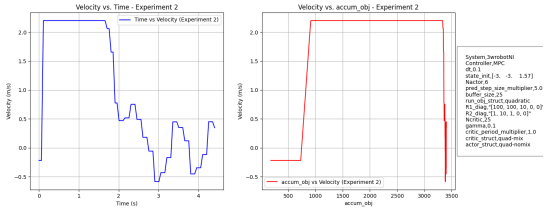


Figure 18: Experiment 3

Lastly, prediction step size was changed and other two factors are constant. Firstly it was set to be 0.1sec and then 1.0. The graphs given below show the relationship between time and velocity performance of the robot.

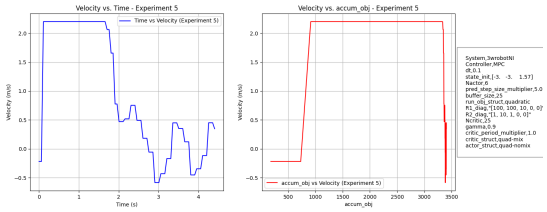


Figure 19: Experiment 5

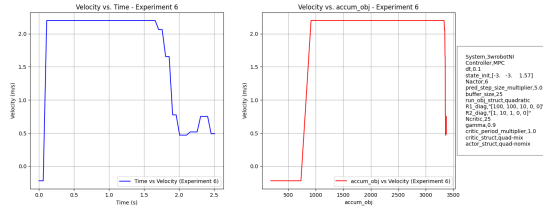


Figure 20: Experiment 6

4.2 Design of Experiment for Nominal Controller

For the Nominal controller another experiment was designed where the initial pose of robot is changed and the tuning parameters are kept constant and then for the next set of experiments initial pose is kept constant and tuning parameters are changes. In total 9 experiments are conducted with following interpretations as shown in the table and 9 graphs are obtained. The performance of the controller is observed by plotting the graphs between velocity and time with the variations. Following values were recorded as in figure 21 shown below:

Experiment #	Pos x	y	z	rho	alpha	beta	Result
1	-3	-3	1.57	0.15	0.17	-0.05	Exp#01
2	-3	-3	1.57	2	14	-1.5	Exp#02
3	-3	-3	1.57	1.5	5	-2	Exp#03
4	-3	-3	1.57	4	7	-3	Exp#04
5	-3	-3	1.57	6	8	-10	Exp#05
6	-1	-1	1.04	0.15	0.17	-0.05	Exp#06
7	-5	-5	0.5	0.15	0.17	-0.05	Exp#07
8	-7	-7	2.09	0.15	0.17	-0.05	Exp#08
9	-10	-10	2.6	0.15	0.17	-0.05	Exp#09

Figure 21: Experimental Design

With the optimal parameters value the following trend is observed as shown in figure 22:

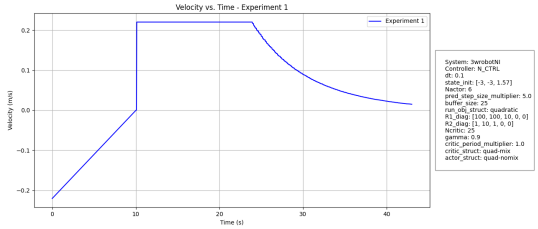


Figure 22: Experiment 1

With the values as shown in the table for Experiment 2 following trend was observed as shown in the figure 23 below:

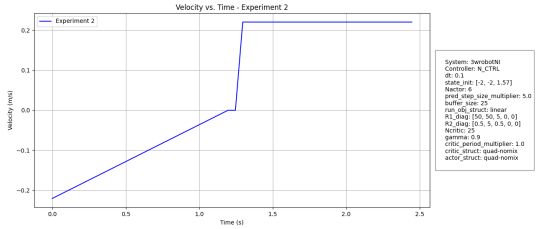


Figure 23: Experiment 2

Similarly, all other experiments were conducted and it was observed that experiment 1 gives a better and smooth performance over the course of time rest of the results that were recorded for all other experiment is available in the appendix.

Now, for the second set of experiment robots initial pose was changed and the control gains are kept constant and following results were obtained:

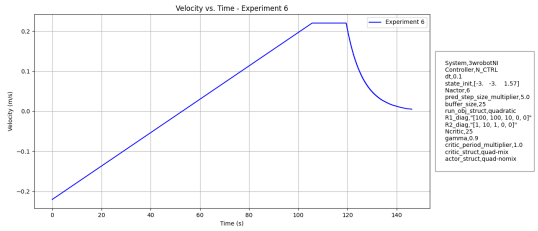


Figure 24: Experiment 6

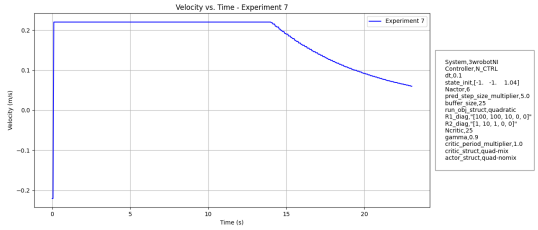


Figure 25: Experiment 7

4.3 Rcognita Simulations

The three controllers are designed and tested in three different environments i.e. rcognita, Gazebo, and real turtle-bot. Let's first analyze the performance of the three controller using rcognita animations.

4.3.1 Nominal Controller

For nominal controller following trajectory is obtained using a differential drive two wheeled robot animation. The given figure 26 shows the behavior of robot with nominal controller. In this case the optimal value of hyper-parameters are selected which are obtained after performing the series of experiments and fine tuning. These parameters are to be learned over the course of time and improves the overall performance of the robot. The control gains for the system are defined as follows:

$$k_\rho = 0.15 \tag{22}$$

$$k_\alpha = 0.17 \tag{23}$$

$$k_\beta = -0.05 \tag{24}$$

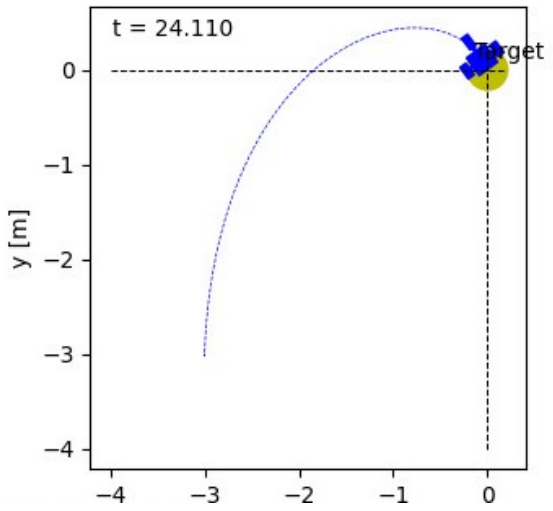


Figure 26: Results in Rcognita

4.3.2 Stanley Controller

For the performance testing of Stanley Controller a car like mobile robot animator inside the rcognita is used. For the Stanley a different trajectory path is also drawn instead of a straight line path to visualize its performance in path tracking and following the trajectory by calculating the heading error and adjusting itself according to the distance measured at each way-points on the trajectory assumed to be the next goal point and update the state of the system. In case of Stanley the tuning parameter is kappa that must be optimized for the better performance of the controller. Therefore, after series of experimentation and hit and trial method the optimal value for the control parameter is selected which gives the better performance and agile trajectory tracking. Following figure 27 shows the best performance of Stanley controller with the trajectory planning and tracking.

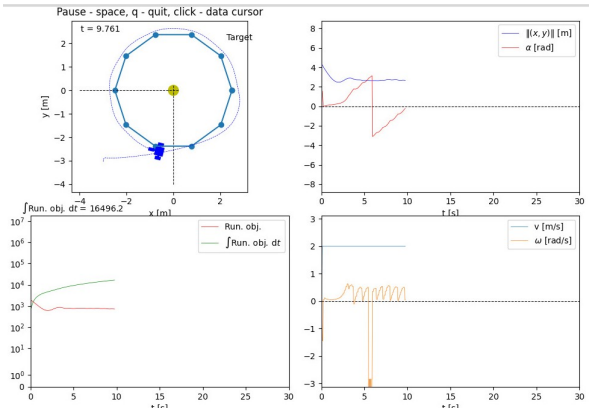


Figure 27: Stanley Controller in Rcognita

Hence, it can be justified that Stanley controllers are well equipped for high speed driving and following the trajectories as compared to nominal controllers.

4.3.3 Model Predictive Controller

Model Predictive Controller is the optimum control solution that is also design in rcognita and tested using the differential drive two wheeled robot animation. The performance metrics of this controller shows the minimal accumulative objective cost and better performance with the optimal control inputs. It follows the shortest path to the goal in lesser time as compared to pure pursuit nominal controller with greater accuracy and speed. The performance behavior can be analyzed by looking into the figure given below:

In this case the optimal set of parameter values is defined as below:

$$\text{sampling\_time} = 0.1 \tag{25}$$

$$N_{\text{actor}} = 6 \tag{26}$$

$$\text{pred\_step\_size} = 1 \tag{27}$$

$$\text{sys\_rhs} = \{\} \tag{28}$$

$$\text{sys\_out} = \{\} \tag{29}$$

$$\text{state\_sys} = \{\} \tag{30}$$

$$\text{buffer\_size} = 20 \tag{31}$$

$$\gamma = 0.9 \tag{32}$$

$$N_{\text{critic}} = 4 \tag{33}$$

$$\text{critic\_period} = 0.1 \tag{34}$$

Therefore, it is observed that at the end of the horizon overall accumulative cost was reduced as compared to the beginning. In the case of Model Predictive Controller two different cost functions are implemented and it is observed that biquadratic function gives slightly better results as compared to quadratic function. The biquadratic function is better as it accounts for the interaction between both states and control inputs that allows the controller to derive better and optimal control strategies. Therefore, it reduces the error, and enhance the stability of the system by providing better representation of cost for the control actions.

4.3.4 MPC with Obstacle Avoidance

For visualizing the behavior of MPC in the presence of obstacles in rcognita, probability density function for multivariate Gaussian distribution is defined to simulate the high cost region that is identified as the big red circle in the middle as obstacle position. The main objective of the integration is to analyze the behavior in the presence of an obstacle and how controller performs to avoid the obstacles. The following figure shows the visualization of obstacles in the environment:

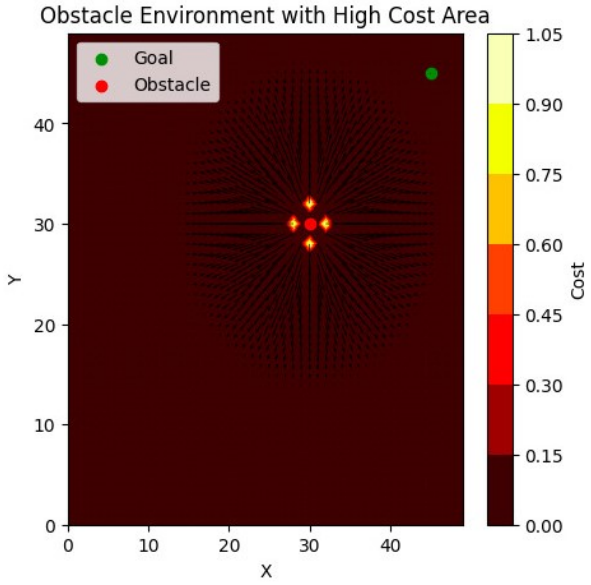


Figure 28: Enviroment with the Obstacles

The following figure 29 shows that MPC tries to avoid the high cost region which means avoiding bumping into the obstacles and chooses the most optimal and shortest path to reach the goal in the minimum time. In the next figure it shows MPC implemented with biquadratic function and it has been seen that the robot does not follow the straight path as it get confused in the presence of obstacles.

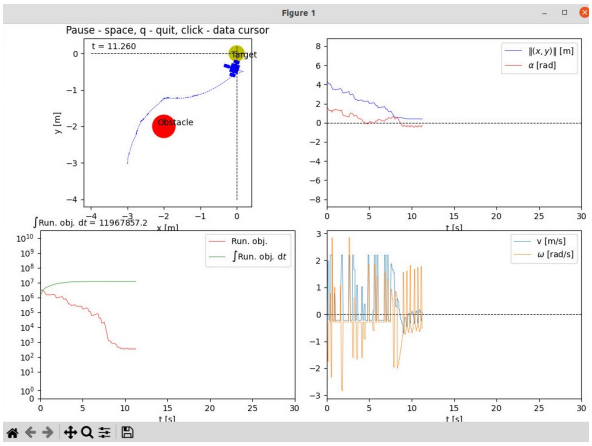


Figure 30: MPC with biquadratic Cost

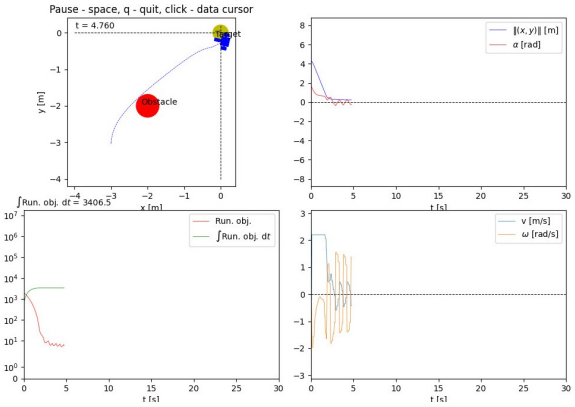


Figure 29: MPC with Obstacle Avoidance

4.3.5 Challenges

In this section lets discuss some of the drawbacks and short comings in the proposed solution which needs further work and experimentation. For the implementation of MPC using obstacle environment, following figure shows the idea behind the environment with obstacles and illustration of how the obstacles looks like in the environment.

In this figure it can be closely observed that the yellow regions showcase the high cost regions and those regions are essentially closer to the place where obstacle is placed. While running the robot with MPC with bi-quadratic cost function inside this environment the accumulative cost is higher as compared to the quadratic function which should be essentially lower. This is due to the adjustment of the penalty factor therefore, the penalty factor can be played around to get the optimal sequence of control inputs. Following figures shows the performance of the controller with different values of the obstacle gain or the penalty.

4.4 Gazebo Simulations

Following section is showing the results of the nominal and model predictive controller results with turtle-bot inside the gazebo environment. The goal position of the robot

in the gazebo is set to be (x, y) = (3, 3). The following figure 31 shows the turtle-bot with nominal controller:

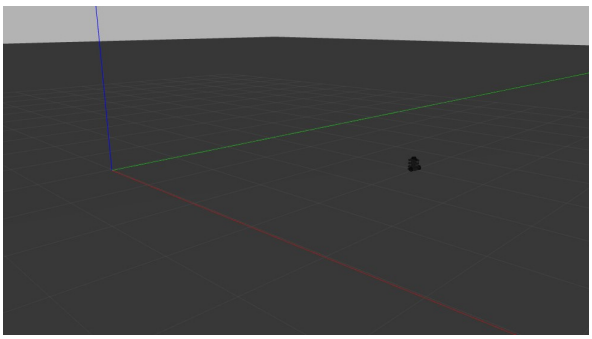


Figure 31: Turtle-Bot with Nominal Controller 1

In the second experiment the goal position is set to be (x, y) = (-5, -5). The position of robot inside the gazebo environment can be visualized in the figure 32 shown below:

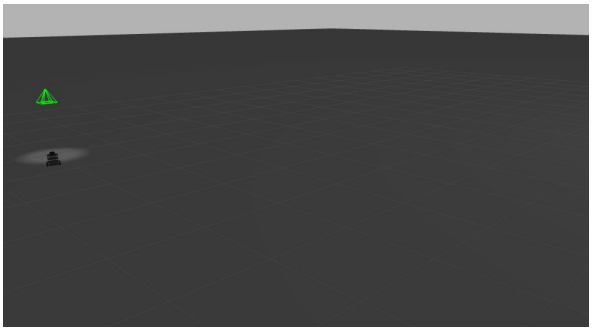


Figure 32: Turtle-Bot with Nominal Controller 2

The turtle-bot with MPC is also visualized in the gazebo environment, shown in the figure 32 below:

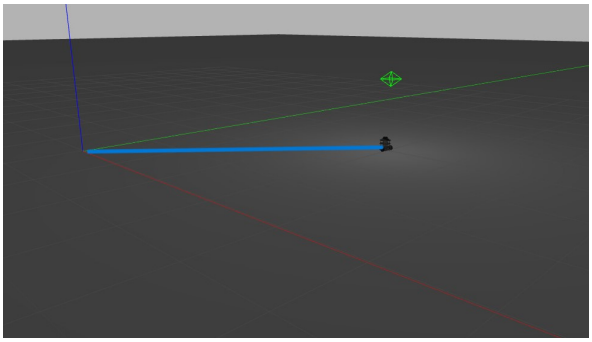


Figure 33: Turtle-Bot with MPC

The video is also recorded which can be seen in the github link given in the appendix.

Real turtle bot is only run with the Nominal Controller and the video was recorded. In the video it can be seen that the goal position is (x,y) = (1, 1)

The turtle-bot is also run with MPC controller but the desired results were not achieved therefore, the code needs further fine tuning. Due to time constraints the

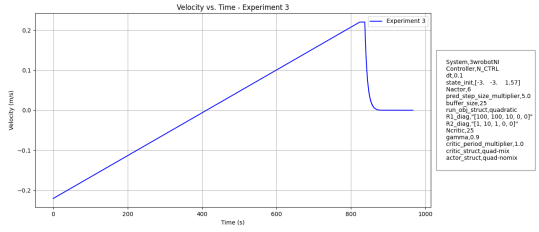
experimentation for this part was not concluded but the partial results are shown in the video available in the github repository.

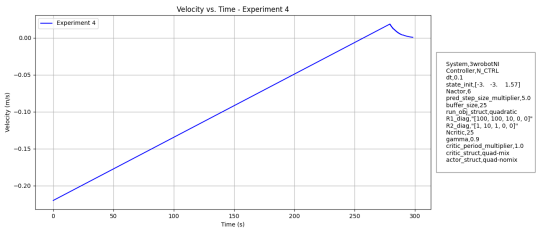
5 Conclusion

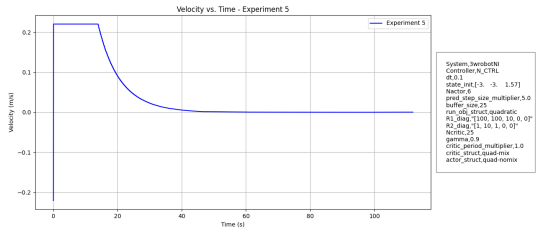
The purpose of this study is to understand the behavior of various controllers on different systems such as differential derive robots and car-like mobile robots. The main objective was to understand the control strategy used by each of the controller Nominal Controller, Stanley Controller, and Model Predictive Controller. The comparison of the results obtained to find out which one is better under which circumstances. It can be observed that different cases require particular control strategy depending upon the requirements and desired results. It has been observed that the most simplest type of controller is Nominal Controller that drives the robot from the initial position to the goal position but it is not very useful when it comes to complex trajectory tracking. Therefore, when the robot has plan its course of motion over the certain period of time, Stanley controller is the best choice. It is most commonly used to drive car-like mobile robots autonomously in complex environments. It is well designed to handle the deviations of the robot from the given trajectory by computing the heading error and correcting the yaw angle to keep robot moving on the desired trajectory. Therefore, whenever more robust handling is required stanley controllers come into play and are the best choice. Stanley controller updates the current state constantly to help robot adjust its pose depending upon the error calculation and velocity value as a control input is included in the control law to adjust the response based on speed. The Stanley controller itself does not simulate or predict vehicle motion rather, it provides a reactive control action based on current errors. Lastly, Model predictive controller belongs to the class of optimal controller which provides the most optimum solution hence, are the most advanced form of controllers whose sole purpose is to minimize the overall cost and localize the robot in unknown environments efficiently and effectively.

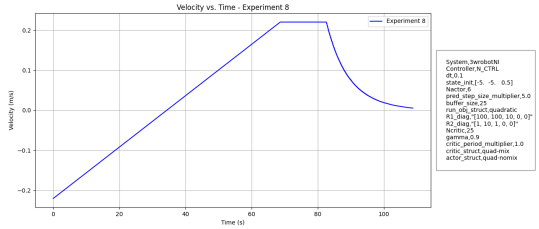
6 Appendix

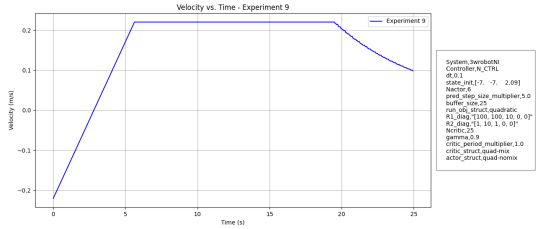
Given below are the remaining results of the experimentation performed for the nominal controller showing the impact of tuning hyperparameters on the performance of the robot.











6.1 Group Member Contributions:

6.1.1 INTRODUCTION, METHODOLOGY, VALIDATION, RCOGNITA:

MANAHIL

6.1.2 ABSTRACTS, SYSTEMS, VALIDATION, EXPERIMENT CODES:

KARINA

6.1.3 CONCLUSION, RESULTS:

YUNUS

6.2 GITHUB LINK

https://github.com/manahilshahid12/rcognita-edu/tree/manahilshahid12-patch-1

References

[1] Kundu, "Understanding Geometric Path Tracking Algorithms — Stanley Controller," Roboquest, Jul. 19, 2020. https://medium.com/roboquest/understanding-geometric-path-tracking-algorithms-stanley-controller-25da17bcc2

[2] “ROBOTIS e-Manual,” ROBOTIS e-Manual. https://emanual.robotis.com/docs/en/platform/turtlebot3/simulation/#gazebo-simulation